

OpenHD FlightLog Studio

3-Schichten-Architektur, Datenbankmodell und Funktionsweise eines Desktop-Tools fuer OpenHD-/MAVLink-Logdateien.

Technologien

C# / .NET 9, Avalonia UI, MySQL, Docker

Architektur

Darstellungsschicht,
Datenverarbeitungsschicht,
Datenhaltungsschicht

Ziel

Rohdaten importieren,
dekodieren, speichern und
auswertbar anzeigen

Datenbank

MySQL `openhdl_flightlog`,
optional per Docker gestartet

1. Projektueberblick

Vom binaeren MAVLink-Log zur strukturierten Datenbankansicht.

OpenHD FlightLog Studio ist eine Desktop-Anwendung zum Analysieren von OpenHD- und MAVLink-Logdateien. Solche Logdateien enthalten rohe binaere Paketdaten. Direkt im Editor sind diese Daten kaum verstaendlich, weil sie aus Headern, Payloads, Checksummen und optionalen Zusatzdaten bestehen.

Das Tool importiert diese Dateien, erkennt MAVLink-v1- und MAVLink-v2-Frames, dekodiert deren Inhalte und speichert die Ergebnisse in einer lokalen MySQL-Datenbank. Danach koennen die Daten in Tabellen angezeigt, durchsucht und teilweise bearbeitet werden.

Zentrale Ziele

- Rohdaten aus Logdateien in strukturierte Daten umwandeln.
- MAVLink-Nachrichten und Felder lesbar machen.
- Importierte Logs dauerhaft in einer relationalen Datenbank speichern.
- OpenHD-spezifische MAVLink-Definitionen verwenden.
- Import-, SQL- und Debug-Schritte nachvollziehbar anzeigen.

Projektdaten

Punkt	Umsetzung
Anwendungstyp	Desktop-Anwendung
UI-Framework	Avalonia UI
Programmiersprache	C# auf .NET 9
Datenbank	MySQL <code>openhdl_flightlog</code>
Datenbankzugriff	<code>MySqlConnection</code>
Datenbankbereitstellung	lokal oder automatisch ueber Docker

OpenHD FlightLog Studio - Dokumentation

2. 3-Schichten-Architektur

Die Software ist nach Darstellung, Verarbeitung und Speicherung getrennt.

Die vorgegebene Architektur teilt die Software in drei Schichten. Diese Schichten muessen nicht als drei getrennte Programme laufen. Entscheidend ist, dass die Verantwortlichkeiten getrennt sind.

Darstellungsschicht

Datenverarbeitung

Datenhaltung

Implementiert die Benutzeroberfläche: Fenster, Buttons, Tabellen, Tabs und Dateiauswahl.

`MainWindow.axaml`
`MainWindow.axaml.cs`

Bereitet Daten auf, verarbeitet Logdateien, dekodiert MAVLink-Pakete und steuert Abläufe.

`MainWindowViewModel.cs`
`FlightLogImportService.cs`

Speichert persistente Daten, führt SQL-Abfragen aus und verwaltet Transaktionen.

`FlightLogDatabase.cs`
MySQL

View
Avalonia GUI



Datenverarbeitung
ViewModel + Services



Datenhaltung
FlightLogDatabase +
MySQL

Schicht	Projektdateien	Aufgabe
Darstellungsschicht	<code>MainWindow.axaml</code> , <code>MainWindow.axaml.cs</code>	GUI, Tabs, Buttons, Tabellen, Dateiauswahl
Datenverarbeitungsschicht	<code>MainWindowViewModel.cs</code> , <code>FlightLogImportService.cs</code> , Parser/Decoder	Importsteuerung, Parsen, Dekodieren, Daten vorbereiten
Datenhaltungsschicht	<code>FlightLogDatabase.cs</code> , MySQL	Schema, SQL-Abfragen, Transaktionen, Speichern und Laden

Die Anwendung ist keine Web-App und benötigt keinen separaten HTTP-Backend-Server. Die Datenverarbeitungsschicht läuft innerhalb der Desktop-App, ist aber im Code getrennt umgesetzt.

3-Schichten-Architektur

3. Umsetzung im Code

Die Klassen sind so getrennt, dass jede Schicht eine klare Aufgabe hat.

Darstellungsschicht

Die View ist die grafische Oberfläche. Sie zeigt Daten an und nimmt Benutzeraktionen entgegen. SQL-Abfragen oder MAVLink-Dekodierung finden dort nicht statt.

- `MainWindow.axaml` : Layout mit Buttons, Tabs, DataGrids, Statusleiste.
- `MainWindow.axaml.cs` : Avalonia-spezifische Dateiauswahl.

Datenverarbeitungsschicht

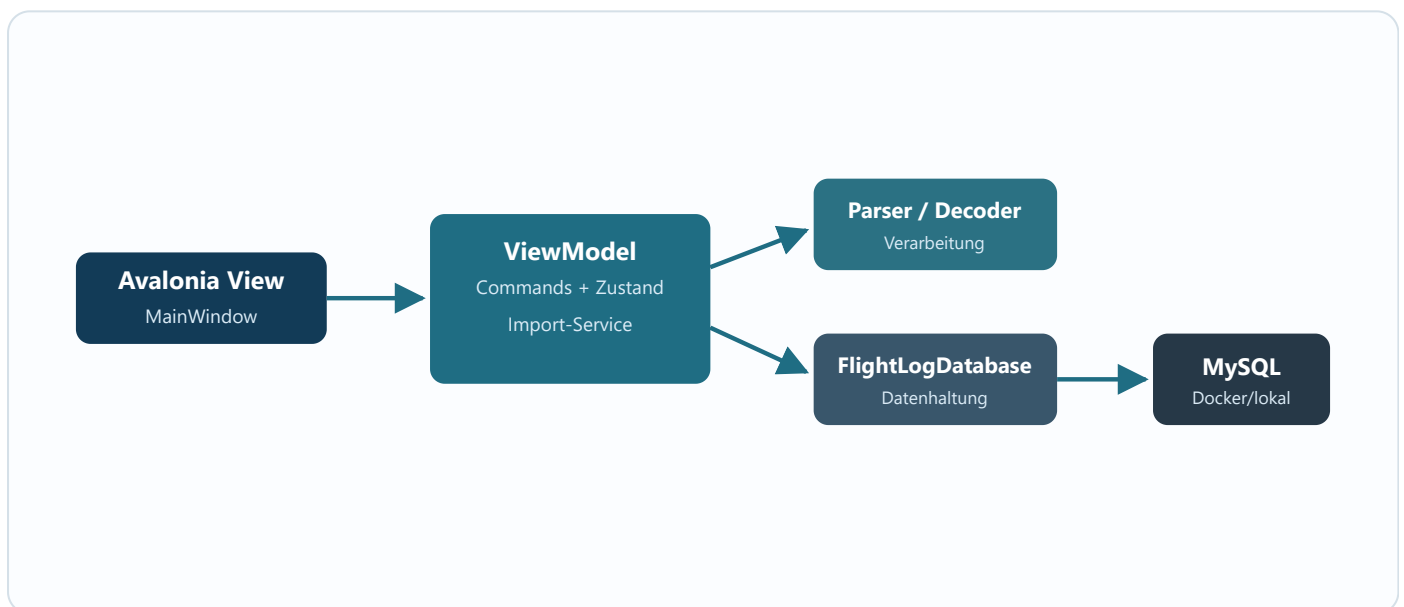
Diese Schicht enthaelt die fachliche Logik. Nach dem Umbau liegt die Importlogik nicht mehr in der Datenbankklasse, sondern im `FlightLogImportService`.

- `MainWindowViewModel.cs` : Commands, UI-Zustand, Auswahlreaktionen.
- `FlightLogImportService.cs` : Datei lesen, Frames parsen, Felder dekodieren, Importdaten vorbereiten.
- `MavlinkParser.cs` : MAVLink-v1/v2-Frames aus Bytes extrahieren.
- `DynamicMavlinkDecoder.cs` : Payloads mit gespeicherten Definitionen dekodieren.
- `OsdReplayService.cs` : OSD-Replay-Daten aus Logvariablen berechnen.

Datenhaltungsschicht

`FlightLogDatabase.cs` ist fuer MySQL zustaendig. Die Klasse erstellt Tabellen, laedt Daten, speichert vorbereitete Importdaten und nutzt Transaktionen.

- Keine direkte Datei- oder MAVLink-Verarbeitung.
- Keine Parser-/Decoder-Aufrufe.
- Speicherung ueber `SaveImportedLog`.



Umsetzung im Code

4. Hauptfunktionen

Das Tool ist Viewer, Parser, Decoder und Datenbankanwendung zugleich.

Import

- Logdateien wie `.oLog`, `.tlog`, `.bin` und `.mavlink` importieren.
- MAVLink v1 mit Startbyte `0xFE` erkennen.
- MAVLink v2 mit Startbyte `0xFD` erkennen.

Anzeige

- Importierte Logs anzeigen.
- MAVLink-Frames tabellarisch anzeigen.
- Dekodierte Felder anzeigen und teilweise bearbeiten.

- Optionale Sidecar-Dateien `.debug.json1` nutzen.

Dekodierung

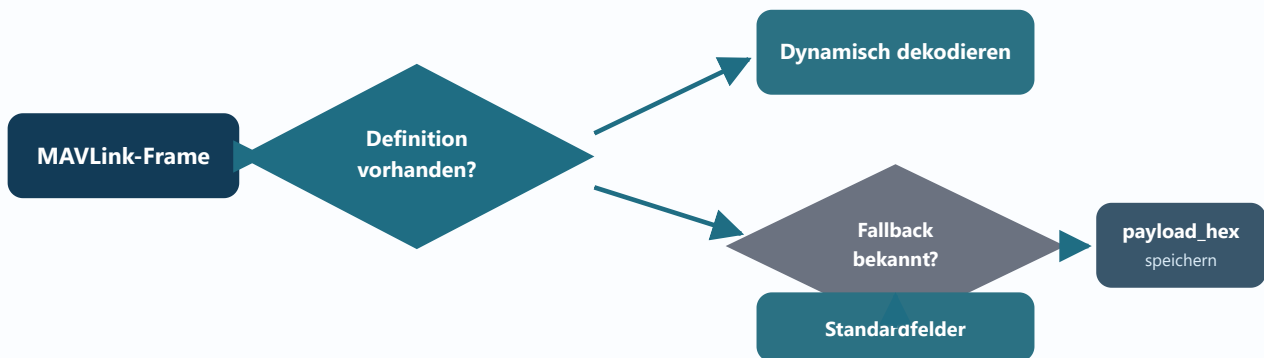
- OpenHD-MAVLink-Definitionen aus Headerdateien laden.
- Payloads anhand von Typ, Arraylaenge und Offset dekodieren.
- Fallback fuer bekannte Standard-MAVLink-Nachrichten.
- Unbekannte Nachrichten als `payload_hex` erhalten.

- Automatische Logvariablen und OSD-Replay anzeigen.
- Debug-Ereignisse fuer Import und SQL anzeigen.

Persistenz

- Logs, Messages, Felder und Definitionen in MySQL speichern.
- Manuelle Variablen und Notizen speichern.
- Definitionen erneut importieren und aktualisieren.

Dekodierlogik



Hauptfunktionen

5. Importablauf

Der Import zeigt das Zusammenspiel der drei Schichten.

- 1 Der Benutzer klickt in der Darstellungsschicht auf

Import
Log

- 2 Die View oeffnet den Dateiauswahldialog und liefert den Dateipfad an das ViewModel.
- 3 Das ViewModel startet den

FlightLogImportService

- 4 Der Import-Service liest die Datei und ruft den

MavlinkParser

- 5 Felddefinitionen und optionale Sidecar-Zeitdaten werden geladen.

6 DynamicMavlinkDecoder

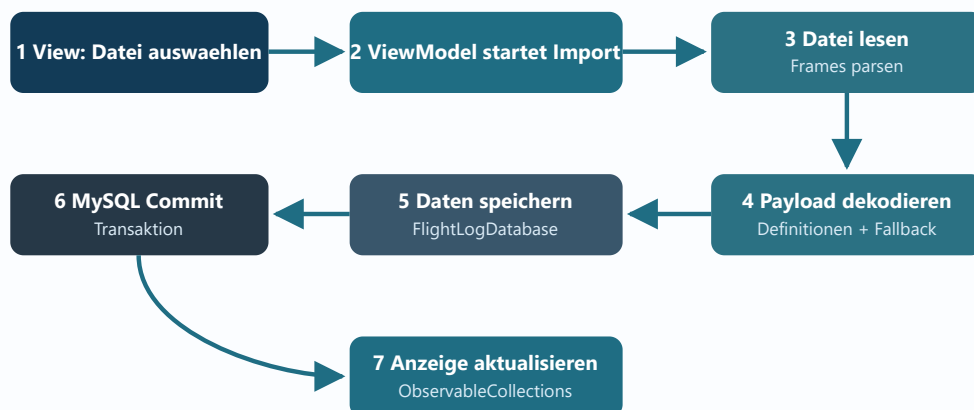
dekodiert
die
Payloads.

- 7 Vorbereitete Importdaten werden an

FlightLogImporter

- 8 Die Datenhaltungsschicht speichert Logs, Messages und Felder in einer Transaktion.

- 9 Das ViewModel aktualisiert die Collections; die DataGrids zeigen die neuen Daten.



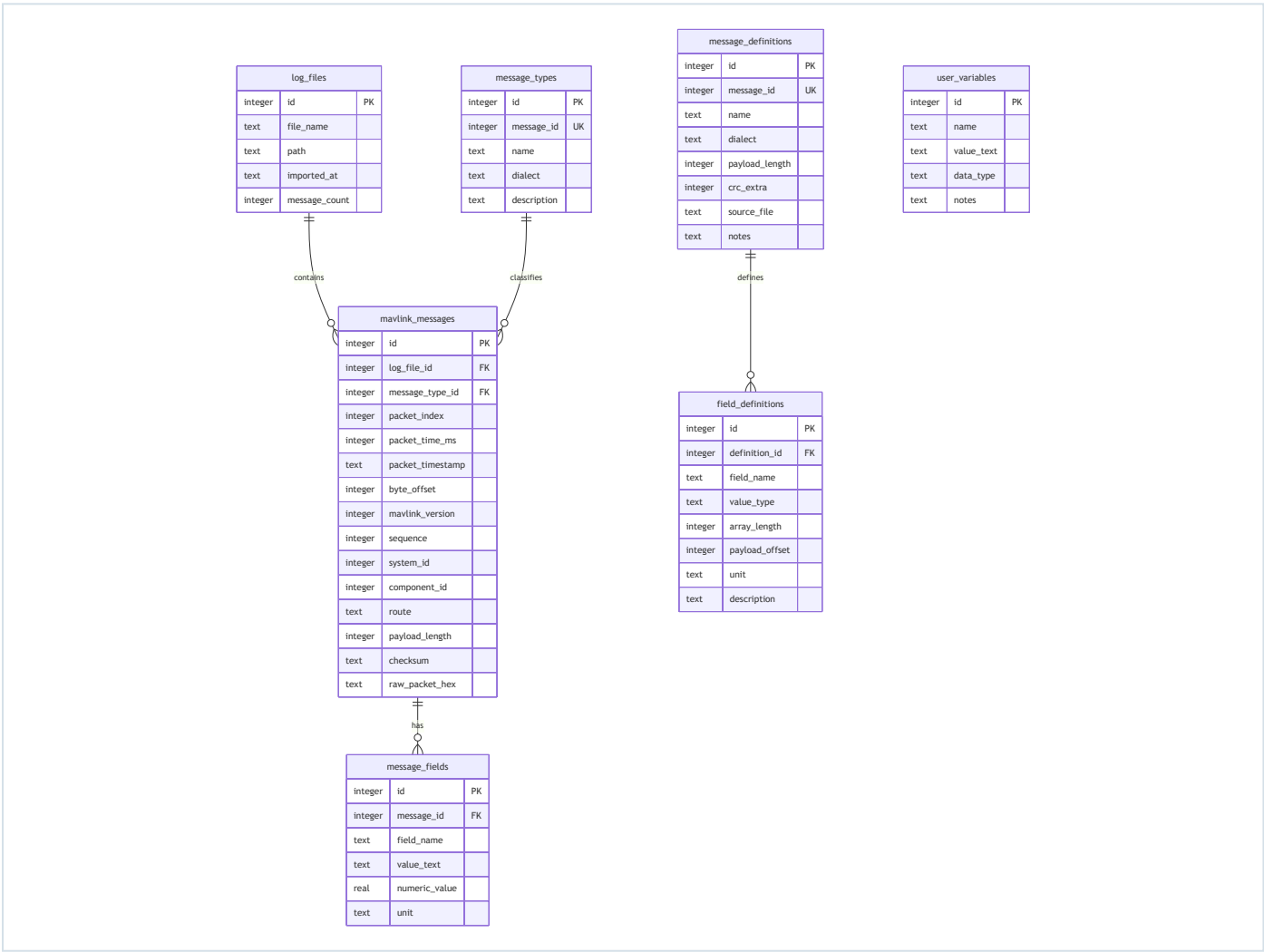
Der Import wird in MySQL als Transaktion gespeichert. Dadurch wird ein Log vollständig oder gar nicht gespeichert. Das verhindert halb importierte Daten.

Importablauf

6. Datenbankmodell

Die Datenbank speichert Logs, Messages, Felder, Definitionen und Notizen.

Tabelle	Zweck
log_files	Ein Datensatz pro importierter Logdatei.
message_types	Normalisierte MAVLink-Message-Typen mit Message-ID, Name und Dialekt.
mavlink_messages	Ein Datensatz pro erkanntem MAVLink-Frame.
message_fields	Dekodierte Felder einer Message.
user_variables	Manuelle Variablen und Notizen.
message_definitions	MAVLink-Message-Definitionen aus OpenHD-Headern.
field_definitions	Feldlayout einer Message-Definition.



Das Diagramm zeigt die Beziehungen zwischen den wichtigsten Tabellen. Die aktuelle Implementierung verwendet MySQL; das Diagramm beschreibt die fachlichen Tabellen und Beziehungen.

7. Datenbankentscheidungen

Konsistenz, Nachvollziehbarkeit und Performance stehen im Vordergrund.

MySQL als relationale Datenbank

Die Daten sind stark strukturiert: Ein Log enthaelt viele Messages, eine Message enthaelt viele Felder, und Definitionen enthalten viele Felddefinitionen. Diese Beziehungen passen gut zu einer relationalen Datenbank mit Primary Keys, Foreign Keys und Joins.

Foreign Keys und Cascade Delete

Foreign Keys sichern Beziehungen zwischen Tabellen ab. Wenn ein Log geloescht wird, entfernt MySQL die zugehoerigen Messages und Felder automatisch ueber `ON DELETE CASCADE`. Dadurch bleiben keine verwaisten Detaildaten zurueck.

Transaktionen

Beim Import werden viele Datensaeetze geschrieben. Eine Transaktion sorgt dafuer, dass ein Import als Einheit behandelt wird. Entweder wird alles gespeichert oder nichts dauerhaft uebernommen.

Indizes

Indizes beschleunigen typische UI-Abfragen, zum Beispiel alle Messages zu einem Log oder alle Felder zu einer Message.

Index-Ziel	Nutzen
<code>mavlink_messages.log_file_id</code>	Messages eines Logs schnell laden.
<code>message_fields.message_id</code>	Felder einer Message schnell laden.
<code>message_fields.field_name</code>	Felder nach Namen filtern oder sortieren.
<code>field_definitions.definition_id</code>	Felddefinitionen einer Message-Definition laden.

Rohdaten bleiben erhalten

Das Feld `raw_packet_hex` speichert das originale MAVLink-Paket als Hex-String. Das hilft beim Debugging und macht die Dekodierung nachvollziehbar.

Datenbankentscheidungen

8. Technologieentscheidungen

Warum Avalonia, Docker und MySQL fuer dieses Projekt sinnvoll sind.

Avalonia statt WinForms

WinForms ist stark Windows-orientiert. Avalonia funktioniert plattformuebergreifend und unterstuetzt moderne XAML-aehnliche Oberflaechen, MVVM und Data Binding.

- Cross-platform: Windows, Linux und macOS.
- Gute Trennung zwischen View und ViewModel.
- Geeignet fuer Tabs, DataGrids und Statusanzeigen.
- Besser passend zu einer geschichteten Desktop-Anwendung.

Dockerized MySQL statt XAMPP

XAMPP ist vor allem fuer PHP-/Webprojekte gedacht und bringt Apache, PHP und weitere Komponenten mit. Dieses Projekt braucht nur MySQL. Docker ist deshalb gezielter und reproduzierbarer.

- Definierte MySQL-Version ueber `mysql:8.4`.
- Definierter Containername `openhhd-flightlog-mysql`.
- Keine manuelle XAMPP-Konfiguration.
- Weniger Konflikte mit anderen lokalen Diensten.
- Einfacher fuer Demos auf anderen Rechnern.

Warum keine HTTP-Web-App?

Das Tool arbeitet mit lokalen Logdateien. Eine Desktop-App passt dazu besser als ein Browser-Frontend mit REST-API. Ein HTTP-Backend wuerde Routing, Hosting und zusaetzliche Infrastruktur erfordern, ohne fuer diesen lokalen Analysefall notwendig zu sein.

Die 3-Schichten-Architektur beschreibt Verantwortlichkeiten. Sie verlangt nicht automatisch einen Webserver.

Technologieentscheidungen

9. Fazit

Das Projekt erfuellt die geforderte Schichtenlogik.

OpenHD FlightLog Studio setzt die 3-Schichten-Architektur innerhalb einer Desktop-Anwendung um. Die Darstellungsschicht ist die Avalonia-Oberflaeche, die Datenverarbeitungsschicht besteht aus ViewModel, Import-Service, Parsern und Decodern, und die Datenhaltungsschicht besteht aus `FlightLogDatabase` und MySQL.

Schicht	Umsetzung
Darstellungsschicht	Avalonia Views: <code>MainWindow.axaml</code> , <code>MainWindow.axaml.cs</code>
Datenverarbeitungsschicht	ViewModel, <code>FlightLogImportService</code> , Parser, Decoder, OSD-Service
Datenhaltungsschicht	<code>FlightLogDatabase</code> und MySQL

Wichtigste Staerken

- Klare Trennung zwischen UI, Verarbeitung und Speicherung.
- Persistente Speicherung in MySQL.
- Import in Transaktionen.
- Foreign Keys und Cascade Delete.
- Flexible MAVLink-Dekodierung durch gespeicherte Definitionen.
- Docker-basierte Datenbankbereitstellung.

Moegliche Pruefungsantwort

Ja, das Projekt folgt der 3-Schichten-Architektur. Die Darstellungsschicht ist Avalonia, die Datenverarbeitungsschicht besteht aus ViewModel, Import-Service, Parsern und Decodern, und die Datenhaltungsschicht besteht aus `FlightLogDatabase` und der MySQL-Datenbank. Die Schichten laufen innerhalb einer Desktop-Anwendung, sind aber nach Verantwortlichkeiten getrennt.